
Using RDF & SPARQL with Stardog

Course Overview

Learning Objectives:

- This course offers specialized training on using RDF and SPARQL within a Stardog environment.
- You will learn to work in the Stardog Studio IDE, on projects that were created and published in Stardog Designer and explored with Stardog Explorer.
- You will learn how to query and update your database using Sparql

Prerequisites:

- Designer Learning Plan
- Studio Overview
- Introduction to RDF & SPARQL or equivalent

Audience: The course is designed for student who wish to become knowledge graph engineers.



Agenda

- Named Graphs from Designer
- Prefixes
- Named Graph Aliases
- Reasoning in Stardog
- Special Named Graphs
- Federated Query Service
 - Querying External Sparql Endpoints (wikidata, dbpedia etc)
- Stored Query Service and Parameterisation



Named Graphs from Designer



Learning Objectives

- Understand and query the Named Graphs created by Designer
- Many ways of finding the IRIs you need to write queries
- Examples of Querying the model Named graph
- Using Explorer's Query Builder to bootstrap your queries



Named Graphs Created by Designer

Projects designed and published using Designer make extensive use of named graphs

A separate named graph will be generated

- for each model
- for each mapped resource

Nothing is written to the default nonamed graph.
Consequently `select * { ?s ?p ?o }` returns no triples.

To find all named graphs you may use the GRAPH keyword e.g.

```
select ?g { graph ?g { }}
```

You can query any named graph using either the GRAPH or FROM keyword

```
select * { graph <tag:stardog:designer:c360:model> { ?s ?p ?o } }
```

or

```
select * from <tag:stardog:designer:c360:model> { ?s ?p ?o }
```

To read every triple in every named graph, as a table, use

```
select ?s ?p ?o { graph ?g { ?s ?p ?o } }
```

Reminder: A Named Graph is a graph that is identified by a unique IRI, allowing it to be distinguished from other graphs in the same dataset.

It enables better organization, lineage and querying

To find all named graphs in the database you may use the GRAPH keyword e.g.

```
select ?g { GRAPH ?g { }}
```

6



Multiple Model Named Graphs from Designer

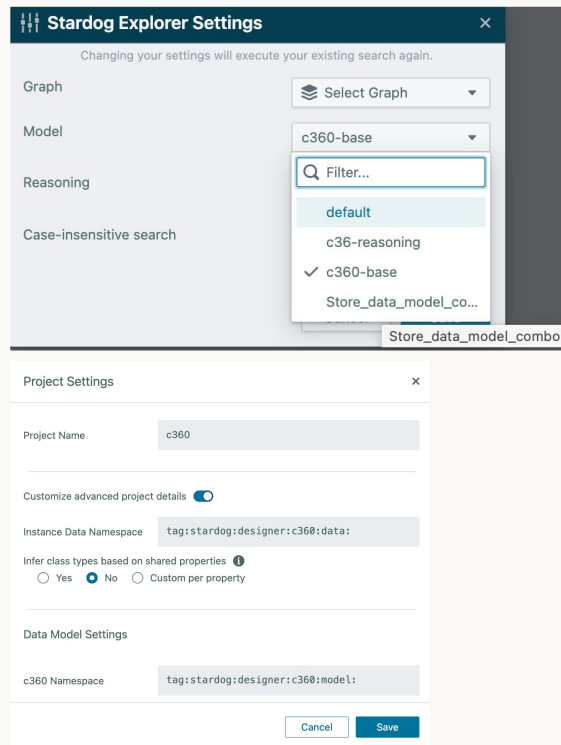
Stardog supports model multi-tenancy. A single database can have support multiple data models. For example, a Basic model might have basic set of classes and properties reflecting the data being modelled. And an Advanced model might have additional classes and properties that are created using rules.

By default, the name of the Named Graph is templated from the project name in Designer. Example: `tag:stardog:designer:c360:model` where `c360` is the project name

Recent versions of Designer feature in-built multi-model support. When using multiple models in a single project, the named graph will be templated to include both the project name and model name. Example: `tag:stardog:designer:Store:c360` where `Store` is the project name and `c360` is the model name

You can always find the names of models Designer has configured in the database by using the drop down in explorer.

Alternatively you can look in the Database properties [schema.graphs](#), detailed later.



The image shows two overlapping dialog boxes from the Stardog Explorer interface. The top dialog, titled 'Stardog Explorer Settings', has a close button (X) in the top right. It contains a warning: 'Changing your settings will execute your existing search again.' Below this are several settings: 'Graph' with a 'Select Graph' dropdown, 'Model' with a dropdown showing 'c360-base', a search bar labeled 'Filter...', and a list of models including 'default', 'c36-reasoning', 'c360-base' (which is selected with a checkmark), and 'Store_data_model_co...'. The bottom dialog, titled 'Project Settings', also has a close button (X). It contains fields for 'Project Name' (set to 'c360'), a toggle for 'Customize advanced project details' (which is turned on), a field for 'Instance Data Namespace' (set to 'tag:stardog:designer:c360:data:'), and radio buttons for 'Infer class types based on shared properties' (set to 'No'). At the bottom, there is a 'Data Model Settings' section with a field for 'c360 Namespace' (set to 'tag:stardog:designer:c360:model:'). 'Cancel' and 'Save' buttons are at the bottom right of this dialog.



Finding the IRIs of Model entities

Before writing queries on the data, it is useful/necessary to identify the data IRIs for Classes, Relationships and Attributes

There are many ways to find the IRIs you need to write a query

1. You can explore the model in Studio, as text
Choose the Models tab and Select your database and model
Use the search function (Ctrl-F or Cmd-F) to find the IRIs needed
2. You can explore the model in Designer.
Navigate to the class, relationship or attribute and expand using `</>`
Eg `[DATA MODEL NAMESPACE]email`, and then find the `DATA MODEL NAMESPACE` from Settings
3. You can find the IRIs from Explorer.
For any model entity (or data instance) you can right click and choose Copy IRI.
4. You can query the model using SPARQL eg To find all classes
`select * from <tag:stardog:designer:c360:model> { ?class a owl:Class; rdfs:label ?classlabel }`
5. Just query the data! You can find IRIs from the data itself using SPARQL
6. Create a query in Explorer. Save the query. Then Open the stored query in Studio



Querying a Model named graph

Since datasets published with Designer always have an explicit model in a Named Graph, it is useful to be able to query the model named graph programmatically.

Here is an example query that queries a named graph `tag:stardog:designer:c360:model` that contains a model

The query returns object properties either with the given class (c360:Purchases) as their domain or range, depending on which condition is met.

The query contains two query pattern blocks that are UNIONed together in the output.

- The first block finds those classes (?range2) where the domain of the property includes the specified class (so:domainIncludes ?class).
- The second block finds those classes (?domain1) where the range of the property includes the specified class (so:rangeIncludes ?class).

The result is a list of ObjectProperties that originate or terminate at c360:Purchases, together with the names of the other classes involved.

find Object Properties in or out of a given class

```
select *  
from  
<tag:stardog:designer:c360:model>  
{  
  bind (c360:Purchases as ?class)  
  {  
    ?obj_prop a owl:ObjectProperty ;  
    so:domainIncludes ?class ;  
    so:rangeIncludes ?range2 .  
  }  
union  
{  
  ?obj_prop a owl:ObjectProperty ;  
  so:domainIncludes ?domain1 ;  
  so:rangeIncludes ?class .  
}  
}
```



Finding your data



Data published from Designer, by default, will be templated using the project name, and the name of the resource from which the data was mapped.



Example: `tag:stardog:designer:c360:data:Purchases` where **c360** is the Designer project name and **Purchases** is the name of the resource that was mapped. (Notice is not templated based on the class the data is mapped to but on the resource it was mapped from.)



You can query the Named graph using the GRAPH or FROM keyword.



Example: `select * from <tag:stardog:designer:c360:data:Purchases> {?s ?p ?o}`



To query for a pattern containing triples from different graphs, you must use FROM keyword.



Example: Find the names of products ordered in quantities ≥ 20 .



Here the **blue triples** come from **Purchases** while the **orange triples** come from **products**. Using GRAPH keyword will search each named graph, in turn, looking for the complete pattern. Since the complete pattern appears in neither graph, the query does not find the pattern. In this case you must use the FROM keyword



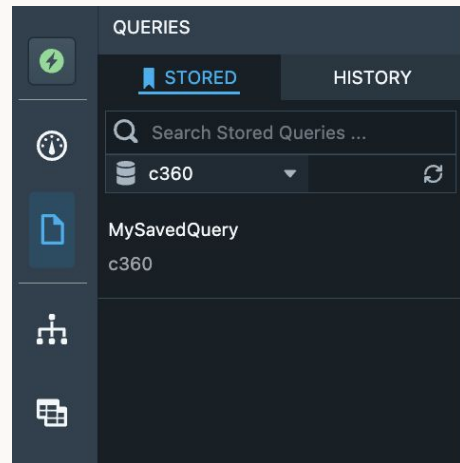
```
# A pattern from multiple graphs: This works
select distinct ?productName
FROM
<tag:stardog:designer:c360:data:Purchases>
FROM
<tag:stardog:designer:c360:data:products>
{
  ?order c360:forProduct ?product .
  ?order c360:quantity ?quantity .
  ?product rdfs:label ?productName .
  filter( ?quantity >= 20)
}
```

```
# This return no results
select distinct ?productName
{ GRAPH ?g
  {
    ?order 10c360:forProduct ?product .
    ?order c360:quantity ?quantity .
    ?product rdfs:label ?productName .
    filter( ?quantity >= 20)
  }
}
```

Query Builder to Template Queries

A great way to bootstrap SPARQL query writing in Stardog is to use Query Builder in Explorer.

- Build a query involving the classes and entities you care about. Then the saved query can be opened in Studio.
- Within Studio choose the worksheet tab,  and select the database and query that was saved.
 - You may have to refresh your browser to see a recently saved query
- Before editing the stored query, it is best practice to copy the query and create a new Studio worksheet tab
- For readability, it is best to rename the variables (?obj_0, obj_1) as soon as they are identified. Selecting a variable and then using the shortcut Cmd-Shift-L (Ctrl-Shift-L)¹ will select all instances of the variable to be renamed.



Enhancing the Stored Query

A Stored Query will not include labels. You will need to add triples to the pattern to extract the labels you require.

For Example: Adding the triple below will extract the label for the entity ?obj_0

```
?obj_0 rdfs:label ?label
```

While Designer encourages you to label instances when mapping, if no label is assigned, Explorer will use the localname as the label. To mimic this behavior in SPARQL you can use the following:

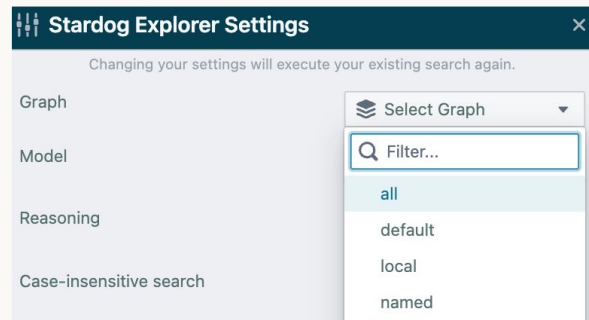
```
optional { ?Obj_0 rdfs:label ?currentLabel . }
```

```
BIND(IF(BOUND(?currentLabel), ?currentLabel, LOCALNAME(?Obj_0)) AS ?newLabel)
```

This will return the label if one is assigned - otherwise returns the localname of the IRI.

Notice that a query built in Explorer will query all of the graphs that were set in Explorer - and this is usually 'all' graphs. A first performance enhancement might be to query only the necessary graph by commenting out the graphs not needed. (The same can be achieved in Explorer by selecting only the graphs needed, in settings as shown, before building the query.)

You are now in a position to add further enhancements to your query.



Prefixes in Stardog

Prefixes and Namespace Recap

- Prefixes are used to shorten full IRIs for legibility and conciseness. Eg `rdf:type`, `rdfs:label`. For Example:
 `e:Customer:5`
 is easier on the eye than
 `<tag:stardog:designer:c360:data:Customer:5>`
- Prefixes are not necessary!!! They make no difference to the underlying data - they are for the convenience of us humans!
- The prefix, in a prefixed IRI, is the string up to the first colon, shown in Orange. The parser will expand any prefixed IRI to build the full IRI. It reads the prefix and replaces it with the text specified in the PREFIX command.

You can choose to prefix a : separated IRI to include any portion you please

```
PREFIX a: <tag:>
PREFIX b: <tag:stardog:>
PREFIX c: <tag:stardog:designer:>
PREFIX d: <tag:stardog:designer:c360:>
PREFIX e: <tag:stardog:designer:c360:data:>
PREFIX f: <tag:stardog:designer:c360:data:Customer:>
PREFIX g: <tag:stardog:designer:c360:data:Customer:5>
```

Using the above prefixes, ALL these refer to the same IRI

```
<tag:stardog:designer:c360:data:Customer:5>
a:stardog:designer:c360:data:Customer:5
b:designer:c360:data:Customer:5
c:c360:data:Customer:5
d:data:Customer:5
e:Customer:5
f:5
g:
```

In all cases, the `localname()` function returns `5` and `namespace()` returns `tag:stardog:designer:c360:data:Customer:`



Prefixes and Namespace Recap (con't)

- Prefixes can be specified in turtle files using `@prefix` and in Sparql using `prefix`
- The namespace of an IRI is the portion up to the last separator. In our example the namespace is `tag:stardog:designer:c360:data:Customer:`
- For compression of Select query output, Stardog will use a prefix that can refer to the Namespace or any portion of the namespace, when the IRI uses only `:` as separator
 - Example: Only prefix `f` refers to the complete namespace of the given IRI



Using Prefixes in Stardog

In this example, `insd` is the prefix.

`tag:stardog:designer:c360:data:Customer:` is the namespace and the localname is `5`.

While we can choose any prefix we like for our own IRIs, there are many namespaces so well established, including their prefixes, that they are always assigned the same prefix. This is only convention.

```
prefix rdf: http://www.w3.org/1999/02/22-rdf-syntax-ns#  
rdf:type
```

Other examples are `rdfs:` `xsd:` `owl:`

Here the prefix prefixes only a portion of the namespace

```
prefix insd: <tag:stardog:designer:c360:data:>  
insd:Customer:5 # here the prefixed component is only part of the  
namespace
```

Namespaces, like IRIs, don't point anywhere or do anything!

A namespace could be a URL that could resolve to a webpage - but there is no requirement that it does.

Example:

```
# define a prefix  
prefix insd: <tag:stardog:designer:c360:data:>
```

```
# prefixed IRI  
insd:Customer:5
```

```
# full IRI  
tag:stardog:designer:c360:data:Customer:5
```

```
select (string(insd:Customer:5) as ?fullIRI)  
      (namespace(insd:Customer:5) as  
?namespace)  
      (localname(insd:Customer:5) as  
?localname) {}
```

16

returns

```
"tag:stardog:designer:c360:data:Customer:5"  
"tag:stardog:designer:c360:data:Customer:"  
"5"
```



Prefix and Namespace Quiz 1

Example IRI `<http://example.com/myPath#id3>`

We recognise it is a full IRI in turtle and Sparql because it is surrounded by `<>`

Q: What is the namespace?

A: `http://example.com/myPath#`

Q: What is the localname?

A: `id3`

Consider prefix `ex: <http://example.com/myPath#>`

Q: How can we write the IRI as a prefixed IRI

A: `ex:id3`

17

Q: Does the prefix correspond with the namespace?

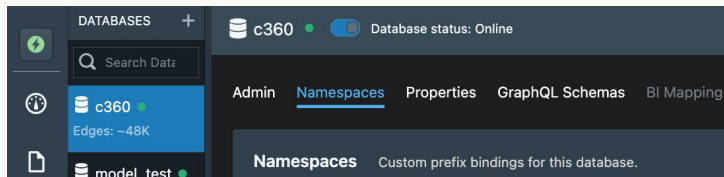
A: Yes. The prefix refers to the entire namespace.



Where to Define the Prefix

Prefixes may be defined anywhere in the worksheet, a turtle file or in the database's metadata. Being defined in the database means we can use the prefix in any turtle file or Sparql worksheet without redefining it.

To define a prefix in the metadata from the databases icon choose the database and choose Namespaces



Metadata defined prefixes are available to all users of the database, everywhere.

Worksheet defined prefixes (that are not in the Metadata) are available only in the worksheet, or turtle file, in which they are defined.

The following prefixes are defined, by default, in every Stardog database created

PREFIX : <<http://api.stardog.com/>>
PREFIX owl: <<http://www.w3.org/2002/07/owl#>>
PREFIX rdf: <<http://www.w3.org/1999/02/22-rdf-syntax-ns#>>
PREFIX rdfs: <<http://www.w3.org/2000/01/rdf-schema#>>
PREFIX so: <<https://schema.org/>>
PREFIX stardog: <tag:stardog:api:>
PREFIX xsd: <<http://www.w3.org/2001/XMLSchema#>>



Prefix and Namespace Quiz 2

Considering the default prefixes show...

What is the prefixed IRI of

<<http://www.w3.org/1999/02/22-rdf-syntax-ns#type>>

Ans: [rdf:type](#)

What is the full IRI referenced by [rdfs:label](#)

Ans: <<http://www.w3.org/2000/01/rdf-schema#label>>

What is the full IRI referenced by the 'blank' prefix [:x123](#)

Ans: <<http://api.stardog.com/x123>>

Is this a prefixed IRI or full IRI [stardog:context:all](#)

Ans: [It is a prefixed IRI. It is not enclosed in <>](#)

To what IRI does it refer?

Ans: <[tag:stardog:api:context:all](#)>

The following prefixes are defined, by default, in every database created

PREFIX : <<http://api.stardog.com/>>

PREFIX owl: <<http://www.w3.org/2002/07/owl#>>

PREFIX rdf:

<<http://www.w3.org/1999/02/22-rdf-syntax-ns#>>

PREFIX rdfs: <<http://www.w3.org/2000/01/rdf-schema#>>

PREFIX so: <<https://schema.org/>>

PREFIX stardog: <[tag:stardog:api:](#)>

PREFIX xsd: <<http://www.w3.org/2001/XMLSchema#>>



Compressing Output Using Prefixes

	Compress IRIs
CONSTRUCT, DESCRIBE	PREFIX in Worksheet
SELECT	PREFIX in Metadata
CONSTRUCT, DESCRIBE, SELECT	PREFIX in Worksheet and Metadata

For convenience it is often desirable to compress or compact (ie use prefixed IRIs)

The output of DESCRIBE and CONSTRUCT, will compress prefixes that are defined in the worksheet.

SELECT queries will compress IRIs only when the prefix is defined in the metadata. It will not compress IRIs defined only in the worksheet

To compress IRIs in all query outputs the prefix must be defined in both the metadata (for SELECT) and in the worksheet (for CONSTRUCT and DESCRIBE)

20

A prefix can be defined only once in a worksheet.

A prefix already defined in the metadata can be defined (or redefined) in the worksheet.

Note: For compression of output from CONSTRUCT or DESCRIBE, the prefix must prefix the entire namespace of the IRI



Named Graph Aliases



Aliasing Named Graphs

As knowledge graphs grew and environments become more complex (e.g., staging, validation, production), hardcoding graph IRIs into SPARQL queries becomes difficult to maintain and update. Named graph aliases offer a solution by acting as intuitive, flexible nicknames for graph IRIs, or collections of graph IRIs, decoupling queries from specific data environments.

- Aliases simplify query maintenance and improve portability across environments. A single alias can point to one, or a collection of named graphs. Your SPARQL query can reference for example `:Customer` and using aliasing you can control which graphs are being referred to. You could point the query at `:Customer_2024` or the collection `:Clients_North` and `:Clients_South`
- Improves maintainability: If the underlying graph IRIs change (e.g., new data sources, renamed regions), you only need to update the aliases — not every query.
- Useful in collaborative environments: Teams can agree on shared aliases that hide complexity and promote consistent graph usage across projects.

22



Aliasing Named Graphs

A Named Graph Alias can act as shortcut or "nickname" for a named graph. For Example:

`:Customers` could be an alias for
`tag:stardog:designer:c360:data:Customers`

A single alias can refer to multiple graphs concatenated. For Example:

`ex:SalesAllRegions` could refer to
`ex:SalesNorth` and `ex:SalesSouth` and `ex:SalesEast`

so that

```
select *  
from ex:SalesNorth  
from ex:SalesSouth from ex:SalesEast {...}  
is equivalent to  
select * from SalesAllRegions {...}
```

Aliases are written to a reserved named graph `stardog:graph:aliases`

Aliases are assigned using a reserved predicate `stardog:graph:alias`

Asserting aliases

```
prefix data: <tag:stardog:designer:c360:data:>  
prefix ex: <http://example.com/>
```

```
insert data {  
  graph stardog:graph:aliases  
  {  
    :customer stardog:graph:alias  
    data:Customers .  
  }
```

```
    ex:SalesAllRegions stardog:graph:alias  
    ex:SalesNorth,  
    ex:SalesSouth,  
    ex:SalesEast .  
  }  
}
```

23



Aliasing Named Graphs Quiz

Aliases must be asserted in a reserved named graph. What is the name of the reserved named graph?

Ans: `stardog:graph:aliases`

To assert aliases, a reserved predicate must be used. What is the reserved predicate?

Ans: `stardog:graph:alias`

Which aliases does the example create?

Ans: `:orders` and `ex:allProducts`

`ex:allProducts` is an alias for which graphs?

Ans: `ex:oldProduct` and `ex:newProduct` concatenated

Fill in the blanks in the following insert data query to give the named graph `:data_2025` the alias `:current`

`insert data { graph blank1 { blank2 stardog:graph:alias blank3 } }`

Ans:

```
insert data { graph stardog:graph:aliases
  { :current stardog:graph:alias :data_2025 }
}
```

Asserting aliases Example

```
prefix data: <tag:stardog:designer:c360:data:>
```

```
prefix ex: <http://example.com/>
```

```
insert data {
  graph stardog:graph:aliases
  {
    :orders stardog:graph:alias data:Purchases
```

```
    ex:allProducts stardog:graph:alias ex:oldProduct .
    ex:allProducts stardog:graph:alias ex:newProduct .
  }
}
```

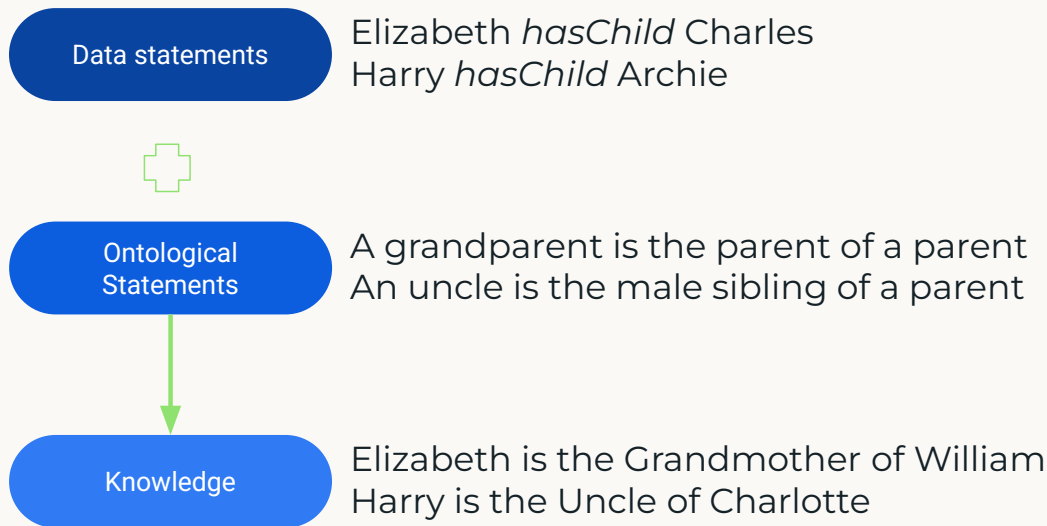
24



Reasoning in Stardog

Review: What is Knowledge

Data + Ontology $\Rightarrow$ Knowledge



“

Knowledge
emerges when
we combine data
with business
logic (ontology)



Reasoning in Stardog

Reasoning (or Inferencing) is the ability to infer new knowledge based on existing data.

It's an essential capability for Knowledge graphs,

- To combine ontological statements from the Data Model or Reasoning Schema or Ontology with triples from the data to create new triples.
- This is the emergence of Knowledge.

Stardog simultaneously supports two reasoning modes:

1. Axiomatic Reasoning like RDFS and OWL
eg `rdfs:subClassOf, owl:SymmetricProperty`
2. Stardog Rules. These are of the form IF...THEN
eg `IF { ?x :hasSpouse ?y } THEN { ?y :hasSpouse ?x }`

The simplest axiomatic rule is `rdfs:subClassOf`

When reasoning is enabled, in the example, `:Bob` and `:Alice` will be a `:Person`

Simple axiom example

```
INSERT DATA {  
  GRAPH <tag:stardog:api:context:schema> {  
    :Person a owl:Class .  
    :Customer a owl:Class ;  
      rdfs:subClassOf :Person .  
    :Employee a owl:Class ;  
      rdfs:subClassOf :Person .  
  }  
}
```

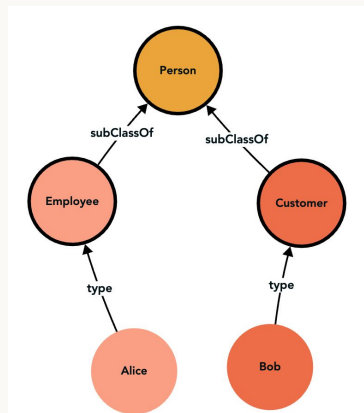
```
INSERT DATA {  
  :Alice a :Employee .  
  :Bob a :Customer .  
}
```

```
SELECT ?x { ?x a :Person }
```

When a data model includes inference rules or axioms, many people would call it an ontology.



Simple Axiomatic Reasoning



Stardog Explorer Settings x

Changing your settings will execute your existing search again.

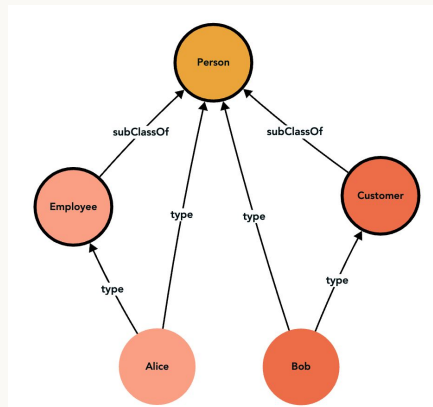
Graph: Select Graph

Model: default

Reasoning: ☐

Case-insensitive search: ☐

Cancel Save



Stardog Explorer Settings x

Changing your settings will execute your existing search again.

Graph: Select Graph

Model: default

Reasoning: ☒

Case-insensitive search: ☐

Cancel Save

Simple axiom example

```
INSERT DATA {  
  GRAPH  
  <tag:stardog:api:context:schema> {  
    :Person a owl:Class .  
    :Customer a owl:Class ;  
    rdfs:subClassOf :Person .  
    :Employee a owl:Class ;  
    rdfs:subClassOf :Person .  
  }  
}
```

```
INSERT DATA {  
  :Alice a :Employee .  
  :Bob a :Customer .  
}
```

```
SELECT ?x { ?x a :Person }
```



More expressive Axiomatic reasoning

Rules and Axioms, just like Data Model triples, can reside in separate Named Graphs to distinguish different sets of rules, to support Data Model Multi-tenancy.

Example: Uncle could be restricted to “Brother of Parent” in one data model or expanded to include “Husband of an Aunt” in a different data model

Rules and Axioms generally reside in the same Named Graph as the Data Model they refer to. The named graph `tag:stardog:api:context:schema` is the named graph that Stardog, by default, will look for reasoning axioms or rules if no other graph is specified.

If the database was created by Designer at publish time, the default will be set to the model published. We will see this in more detail later.

The example makes `:hasSpouse` a symmetric property and makes `:hasPatient` and `:hasDoctor` a symmetric pair of properties.

More expressive Axiom examples

```
insert data {  
  graph  
  <tag:stardog:api:context:schema>  
  {  
    :hasSpouse a  
      owl:SymmetricProperty .  
    :hasPatient owl:inverseOf  
      :hasDoctor .  
  }  
}
```

```
insert data { :Jack :hasSpouse :Jill }
```

```
select * { ?x :hasSpouse ?y }  
# inferred :Jill :hasSpouse :Jack
```

```
insert data { :Mary :hasPatient :Jane }
```

```
select * { ?x :hasDoctor ?y }  
# inferred :Jane :hasDoctor :Mary
```



Reasoning schema Database Properties

Admin Namespaces **Properties** GraphQL Schemas BI Mapping

Properties

Q schema.graphs

Properties modified from their default are indicated with a blue line.

Schema Graphs (reasoning.schema.graphs)

Determines which, if any, named graph or graphs contains the schema (ontology, "TBox") part of the data. The legal value is a comma-separated list of named graph identifiers. The default value is tag:stardog:api:context:schema.

tag:stardog:api:context:schema

Schemas

Reasoning Schemas (reasoning.schemas)

Option to specify the schemas and the named graphs that constitute each schema. The value is a comma-separated collection of schema=IRI pairs. There should be one pair for each named graph in a schema. The graphs for the default schema are set via the reasoning.schema.graphs option.

mod1 = https://example.org:myModel_1, mod2 = https://example.org:myModel_1

Two database properties control the Data Models that are made available in Explorer

In Properties, search for schema.graphs.

1. **Schema Graphs** is the 'Default' named graph in the Explorer Data Models dropdown list.

It will default to **tag:stardog:api:context:schema** if made with "create database" and if made by publishing from designer it will be the first model written.

2. **Reasoning Schemas** is a list of named data models that will appear in the Explorer drop down list.

It a comma separated list of **model=named_graph** pairs

In the example shown, the Schema Graph (or default) is **tag:stardog:api:context:schema**

There are 2 additional models named **mod1** and **mod2** , assigned to particular named graphs



Multiple Reasoning Schema Models

```
insert data {  
  graph ex:myModel_1  
  { :hasPatient owl:inverseOf :hasConsultant }  
}
```

```
insert data {  
  graph ex:myModel_2  
  { :hasPatient owl:inverseOf :hasMD }  
}
```

must set Reasoning Schema to
m1=http://example.org:myModel_1,
m2=http://example.org:myModel_2

```
select * { :Jane ?p :Mary }
```

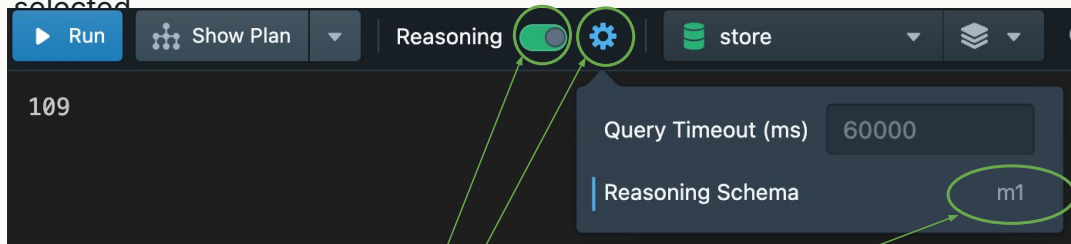
Choosing the reasoning schema gives different results

- Reasoning off: no results
- default gives :hasDoctor
- m1 gives gives :hasConsultant
- m2 gives gives :hasMD

The result of

```
select * { :Jane ?p :Mary }
```

Will have different results based on which reasoning schema is selected



Toggle reasoning on

Click setting

Select reasoning schema



Rule Based Reasoning

Stardog provides a very powerful way of creating rules using the Stardog Rule Syntax (SRS)

SRS rules express reasoning in a procedural way (rather than RDF/OWL declarative way)

Abstract syntax is

IF { condition } THEN { consequence }

Stardog rules are:

- more expressive
- far easier to write/read/test/maintain
- less computationally expensive
- support arithmetic, comparisons and aggregations
- negation



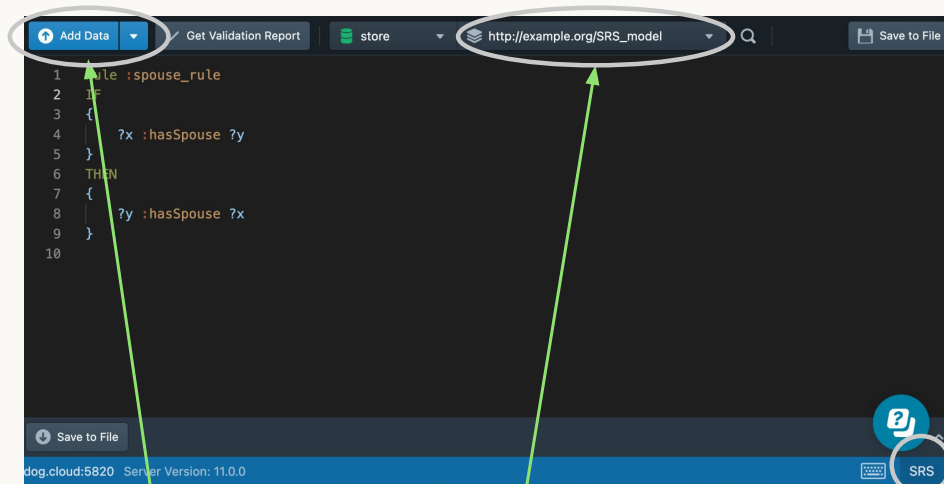
Entering a Rule by Hand

- Open a new tab
- Select SRS as the file type
- Enter the rule as:

RULE <iri for the rule>

```
IF
{
    condition
}
THEN
{
    consequences
}
```

- Enter a named graph
- Check that you have the correct database
- Select Add Data



Select Add Data

Choose named graph for your rule

Select SRS as the file type



Reasoning in Stardog

Stardog Rules supports almost all SPARQL functions including user-defined functions.

- Example 1 Calculates persons :age
- Example 2 Uses that inferred :age to assign the person as an :Adult

Designer Now Supports editing rules in the RULE dialog

Rule Builder [Text Editor](#)

```
IF {
  ?person <http://api.stardog.com/dob> ?birthDate
  bind(year(now()) - year(?birthDate) AS ?age)
}
THEN {
  ?person <http://api.stardog.com/age> ?age
}
```

Example 1: Several SPARQL functions to calculate age

```
IF {
  ?person :birthDate ?birthDate
  bind(year(now()) - year(?birthDate) AS ?age)
}
THEN {
  ?person :age ?age
}
```

Example 2: arithmetic filter on an inferred attribute ?age

```
IF {
  ?person a :Person ;
  :age ?age.
  FILTER (?age >= 18)
}
THEN {
  ?person a :Adult
}
```



Reasoners in Stardog

Stardog Reasoners

As of version 9.0, Stardog comes with two different reasoner implementations, both providing query-time reasoning capability with some differences:

- Blackout is the more mature reasoner implementation that supports more of the RDFS and OWL specifications but has limitations with respect to user-defined rules.
- Stride (alpha) is the next-generation reasoner implementation that supports more expressive user-defined rules, including negation and aggregation but a smaller subset of RDFS and OWL.

Users can switch between the two reasoner implementations by setting the database configuration option `reasoning.stride` to true or false.

The default value for this option is false, which means the Blackout reasoner will be used. No other changes are required after setting this option, and the corresponding reasoner will be used automatically behind the scenes.



Exercises



Special Named Graphs



Special Named Graphs

Stardog includes aliases for several commonly used sets of named graphs. These are:

- **stardog:context:default** - the default (no) context graph
- **stardog:context:named** - all local named graphs, excluding the default graph
- **stardog:context:local** - all local graphs - the default graph and named graphs
- **stardog:context:virtual** - all virtual graphs
- **stardog:context:all** - all local and virtual graphs

These non-standard extensions may be used when specifying the dataset for a query, that is, in the FROM and FROM NAMED clauses of a SPARQL query.

These aliases may also be used via the SPARQL, CLI, API, Pystardog etc

These aliases are different from user defined aliases describe earlier. They are not user editable!

Examples

Query all named graphs but not the nonnamed graph

```
select * from stardog:context:named  
{ ?s ?p ?o }
```

access all the named graphs AND the nonnamed graph

```
select * from stardog:context:local  
{ ?s ?p ?o }
```



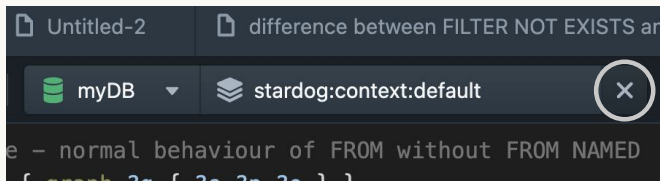
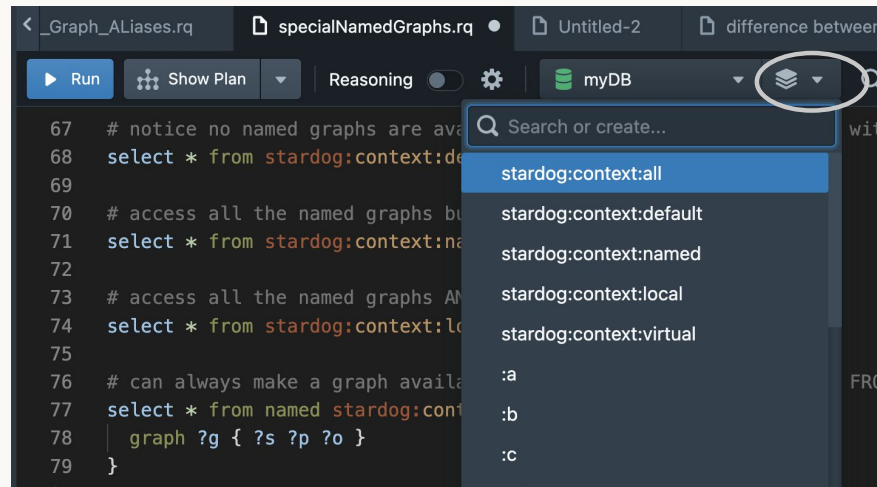
Special Named Graphs

In Studio you can set the context for your query using the drop down next to the database name

Note: Once a special named graph has been selected, the query you run will be run against that aliased set of graphs - you do not need FROM and the special graph name.

Example: Selecting `stardog:context:named` from the drop down and running: `select * { ?s ?p ?o }` is equivalent to making no selection in the drop down and running: `select * from stardog:context:named { ?s ?p ?o }`

Note: to unselect a special named graph, you must hover over the down arrow and select x



Special Named Graphs – Sparql Updates

Do not use special named graphs with Sparql Updates!

- A new named graph, sharing the name of the Alias may be created if updates are made while a Special Named graph is selected
- Results may not be as expected and perhaps may be confusing

Using Special Named Graphs in Studio

- Great for convenience
- Not an alternative to understanding how to use FROM, FROM NAMED and GRAPH correctly.

As part of a query development workflow, use the Studio Special Named Graph Dropdown sparingly.

- History does not retain which Special Named Graphs were selected for the query.
- Stored Query does not retain which Special Named Graphs were selected for the query.
 - Which special named graph must be noted in comments
 - Better just to use FROM and FROM NAMED



Federated Query Service



What is a Federated Sparql Query

A federated sparql query is query that retrieves data from multiple distributed data sources in a single operation.

Eg your sales data in one db and manufacturing data in another. A single query can fetch or join data from both sources

These sources can be ANY sparql compliant endpoint eg

- another Stardog database on your endpoint
- another Stardog database at a different endpoint
- Wikidata <https://query.wikidata.org/sparql>
- Dbpedia <http://dbpedia.org/sparql>
- Biomedical <https://sparql.uniprot.org/sparql>
- Government <https://api.parliament.uk/sparql>

SPARQL-compliant endpoints exist for many domains of research or inquiry



Federated Query Service

When the Stardog endpoint is not the local server you must specify the sparql endpoint more fully

eg for Stardog <http://<server>:<port>/<db>/query>

```
select * {  
  SERVICE <https://sd-8be53c19.stardog.cloud:5820/catalog/query>  
  { graph ?g { ?s ?p ?o } }  
}
```

To query with reasoning enabled, append **/reasoning** to the endpoint URL

```
select * {  
  SERVICE <https://sd-8be53c19.stardog.cloud:5820/catalog/query/reasoning>  
  { graph ?g { ?s ?p ?o } }  
}
```

Note the above query works using the CLI and in Apps via the API. It may not work in Studio.



Service keyword

To submit the query to another database on this endpoint use

SERVICE <db://{database_name}>

```
select * {  
  SERVICE <db://catalog>  
  { graph ?g { ?s ?p ?o } }  
}
```

The <db://{database_name}> is a Stardog Syntax for convenience when accessing databases on the same endpoint

To query other other Sparql endpoints we need the full url of the endpoint

```
select *  
{  
  service <https://sparql.uniprot.org/sparql>  
  { ?protein a up:Protein ; rdfs:label ?proteinLabel }  
} limit 10
```



Example: Wikidata Query Service

To query an external endpoint, if the endpoint has a Sparql Query editor, use that until you're happy with your query.

eg <https://query.wikidata.org/> is the wikidata query editor.

Test your query there.

```
SELECT ?item ?itemLabel
{ ?item wdt:P31 wd:Q146. # Must be a cat
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". } }
```

Then copy the query as a subquery, directly into your Stardog query with the SERVICE keyword

```
select *
{ SERVICE <https://query.wikidata.org/sparql>
  {
    SELECT ?item ?itemLabel
    { ?item wdt:P31 wd:Q146. # Must be a cat
      SERVICE wikibase:label { bd:serviceParam wikibase:language "en". } }
  }
}
```



Stored Query Service



Federated Query Service Wrap Up

Federated queries across databases and external SPARQL endpoints is a powerful capability in Stardog:

- One query, many sources – Combine internal and external SPARQL endpoints in real time.
- Live enrichment – Bring in up-to-date facts without duplicating data.
- Reasoning across sources – Derive new insights spanning multiple datasets, including reasoning on the remote Stardog databases
- No heavy ETL – Query data where it lives, reducing pipelines and storage. No need to manually extract and merge datasets — the federation engine handles it.



Stored Query Service

Stardog's Stored Query Service (SQS) is a powerful feature that allows users to manage, store, and execute SPARQL queries efficiently.

- **Centralized Query Storage:** Store complex queries for easy, shared access and execution.
- **Parameterisable Queries:** Execute pre-defined queries across multiple applications without rewriting them in each app. Pass variables back and forth.
- **Enhanced Performance:** Optimize query execution by leveraging stored queries (that will be precompiled with cached execution plan)
- **Security and Access Control:** Control access to stored queries, ensuring sensitive data is protected.
- **Seamless Integration:** Integrate SQS with external applications using a REST API or CLI. Eg
`$ stardog query execute -u username -p password myDB myStoredQuery`

```
# Simple Stored Query example  
# Given radius calculate circumference  
and area
```

```
select *  
{  
  bind( 2*pi()*?radius as ?circumference )  
  bind( pi()*?radius*?radius as ?area )  
}
```



Stored Query

We've seen earlier that Queries saved from Explorer appear in the Stored Query list, and can be used to bootstrap query writing

Store Queries can be limited to one database or available to the entire instance.

To invoke a stored query use SERVICE <query://{query_name}>

To specify the projected variables from the stored query use the special predicate

```
prefix sqs: <tag:stardog:api:sqs:>
[] sqs:vars {?variable1}, {?variable2}
```

To parametrise your stored query use the special predicate

```
[] sqs:inputs ?input_variable
```

Calling a Stored Query example

```
select * {
  values ?radius { 2 }
  SERVICE <query://{circle}>
  {
    [] sqs:vars ?circumference, ?area ;
    sqs:inputs ?radius
  }
}
```



Saving and Running a Stored Query

In a new Tab, name the tab countTriples, enter your query. EG

```
select (count(*) as $numTriples)
from stardog:context:local # explicit scoping
{ ?s ?p ?o . }
```

- Click on Save and name it (say, 'countTriples')
- Under Advanced, select allow execution against → all database
- If you now select the query in the left panel it will open a new tab with the query for you to run.
- In this case, we specify that we want the query to run against every graph.
- We don't have to scope the stored query explicitly. It will use the scope of the calling query.



Renaming variables?

Suppose we have the following query stored as **t1**

```
select * {  
  values (?a ?b ?c) {( 1 2 3)}  
}
```

We can call the query using

```
select * {  
  SERVICE <query://t1>  
  { [] sqs:vars ?c, ?a, ?b } # names correspond with stored query  
}
```

But if we wish to rename the variables ?a → ?apple, ?b → ?ball

```
select * {  
  SERVICE <query://t1> { [] sqs:var:a ?apple ;  
                           sqs:var:b ?ball ;  
                           sqs:vars ?c }  
}
```

The reserved predicate **sqs:vars** indicates that the triples objects are variable names used in the stored query that should be projected back without name change

The reserved predicate **sqs:var:a** indicates that the variable named **a** in the stored query should be renamed to **?ball** in the outer query



Calling Stored Query Quiz?

Suppose we have the following query stored as **fullName**

```
select * {  
    bind(concat(?firstName, " ", ?lastName) as ?fullName)  
}
```

Q1 We can call the query using - fill in the blanks

```
SELECT * {  
    ?person :firstName ?firstName ; :lastName ?lastName  
    SERVICE <query://[ ]> { [] sqs:vars [ ] ;  
        sqs:inputs [ ] [ ]  
    }  
}
```

Q2 We can rename the variables ?fullname → ?Name

```
select * {  
    SERVICE <query://fullname> { [] sqs:var:fullName ?Name ;  
        sqs:vars ?familyName ;  
        sqs:inputs ?firstName, ?lastName }  
}
```



Correlated Subqueries

A correlated subquery is a query nested inside another query, where the inner query can access variables from the outer query.

Unlike a simple subquery (which is independent), a correlated subquery is evaluated once per binding of the outer query.

Stardog's Stored Query Service makes correlated subqueries cleaner, reusable, and more efficient by letting you store queries and call them with bindings from the outer query.

Consider why this fails # uncorrelated queries

```
select * {  
  values ?radius { 2 3 4 }  
  {  
    bind( 2*pi()*?radius as ?circumference ) # ?radius here in unbound  
  }  
}
```

The outer query binds 2, 3, 4 to ?radius
The inner query does not have the outer ?radius in scope and so ?radius and ?circumference are unbound
The resulting join (there is no join!) produces no bindings

And why this works # correlated subquery

```
select * {  
  values ?radius { 2 3 4 }  
  SERVICE <query://circle> { [] sqs:vars ?circumference, ?area ;  
    sqs:inputs ?radius }  
}
```

Works in a special way.
The outer query binds 2, 3, 4 to ?radius
The service query calls the circle and passes the ?radius values ONE AT A TIME
Question: How many times is 'circle' called?



Review



Review

- Named Graphs from Designer
- Prefixes
- Named Graph Aliases
- Reasoning in Stardog
- Special Named Graphs
- Federated Query Service
 - Querying External Sparql Endpoints (wikidata, dbpedia etc)
- Stored Query Service and Parameterisation



Support Resources

- Self Help:
 - Documentation: <https://docs.stardog.com>
 - Community Portal <https://community.stardog.com>
 - Stardog Academy <https://academy.stardog.com>
- File a Ticket:
 - Via Email: support@stardog.com



Questions



Thank You!



